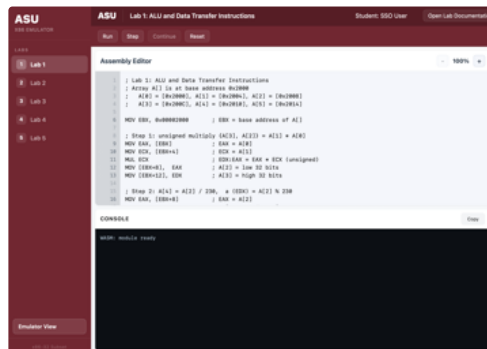
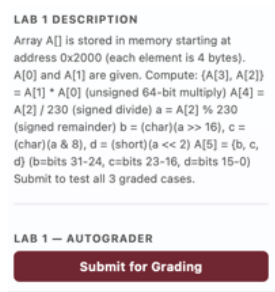


User Documentation

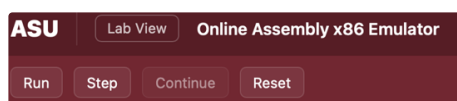
Lab Development and Implementation Guide:



The project includes five progressive labs designed to build x86 assembly skills step by step. Lab 1 covers ALU and data transfer instructions, performing arithmetic operations such as multiply and divide and packing results into memory. Lab 2 focuses on loops using conditional and unconditional branch instructions to remove all occurrences of value from an array. Lab 3 introduces single procedure calls with CALL and RET, implementing two helper functions to construct a new array where each element equals $A[i]$ raised to the power i . Lab 4 covers nested procedure calls through a linked list node insertion algorithm using a two level call tree. Lab 5 implements recursive procedure calls using a shift and add multiplication algorithm without MUL or IMUL.



Each lab includes a built-in autograder that provides instant scoring feedback located at the bottom right of the page. To test your code without submitting, click the Run Button in the top bar – this executes your program and updates the register and memory view in real time. Once satisfied with your result, click the Submit for Grading Buon to receive an instant score. After completing a lab, you can navigate to the next one by clicking the lab buttons on the left sidebar.





Navigation between labs and the free emulator is handled through the Emulator → button at the bottom of the sidebar and the ← Labs button in the emulator's top bar, allowing seamless switching between guided lab work and open-ended assembly exploration.

Debug Panel Description:

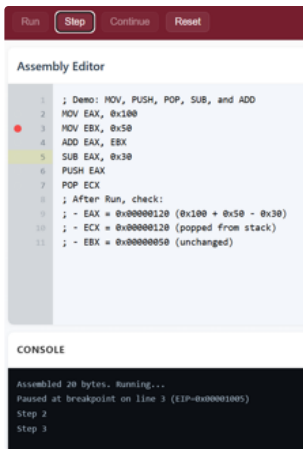
User can toggle one or more breakpoints in their emulator programs to analyze runtime behaviors, register values and stack trace to debug errors.



Run: Starts program execution. Steps through each line and stops at first breakpoint or program end. Must hit Reset to use Run again.



Step: Once breakpoint is hit, Step moves the program execution forward line by line, stopping if another breakpoint is hit or if program ends.



Continue: Once breakpoint is hit, Continue starts program execution beginning at the breakpointed line. Stops at next breakpoint or program end.

Reset: Resets runtime of program, clears all breakpoints, console, and updated register/memory/flag values.

Breakpoint Gutter: Toggle multiple breakpoints, view current execution line during Step.



Registers: The supported registers are EIP, EAX, EBX, ECX, EDX, EBP, ESP, ESI, and EDI, displayed in 32-bit hex format. The registers and memory can be edited directly in UI. The flags shown in UI are ZF, SF, OF, CF, DF, and PF.

REGISTERS	
EIP	0x00001000
EAX	0x00000078
EBX	0x00000000
ECX	0x00000000
EDX	0x00000000
EBP	0x00f00000
ESP	0x00f00000
ESI	0x00000000
EDI	0x00000000

Memory Representation: Memory is shown as a byte window starting at the base address selected by the user. Users can adjust the window size and starting value.

MEMORY @ 0x00001000 (48 bytes)

START BYTES

B8	00	01	00	00	BB	50	00
00	00	01	D8	81	E8	30	00
00	00	50	59	00	00	00	00
00	00	00	00	00	00	00	00
00	00	00	00	00	00	00	00
00	00	00	00	00	00	00	00

Supported Instruction Set:

Data/Stack: MOV, PUSH, POP

Arithmetic/Logic: ADD, SUB, CMP, AND, OR, MUL, DIV, CDQ

Shifts: SHL, SHR, SAR

Control flow: JMP, CALL, RET

Stack Representation: The stack is represented relative to ESP, and stack related instructions update both ESP and the stack view in real time.

Assembly Editor

```

1 ; Demo: MOV, PUSH, POP, SUB, and ADD
2 MOV EAX, 0x100
3 MOV EBX, 0x50
4 ADD EAX, EBX
5 SUB EAX, 0x30
6 PUSH EAX
7 POP ECX
8 ; After Run, check:
9 ; - EAX = 0x00000120 (0x100 + 0x50 - 0x30)
10 ; - ECX = 0x00000120 (popped from stack)
11 ; - EBX = 0x00000050 (unchanged)

```

STACK (top at ESP)

0x00f00000	0x00000000
0x00f00004	0x00000000
0x00f00008	0x00000000
0x00f0000c	0x00000000
0x00f00010	0x00000000
0x00f00014	0x00000000
0x00f00018	0x00000000
0x00f0001c	0x00000000

